

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Artificial Intelligence and Machine Learning

Analytical codes (10 QUESTIONS)

Scenario:

Complex Scenario-Based Problem: Building a Machine Learning Algorithm

An e-commerce company receives 10 million customer transactions every month. The company wants to build a machine learning system that predicts whether a customer will purchase a product within the next 7 days after viewing it.

The available dataset contains the following features:

- * Customer age
- * Gender
- * Location
- * Number of products viewed
- * Time spent on each product page
- * Previous purchase history
- * Cart abandonment rate
- * Device type
- * Payment method
- * Product category
- * Discount offered
- * Purchase outcome (Yes/No)

However, the dataset contains 15% missing values, duplicate records, categorical features, and the classes are imbalanced (only 8% of customers make a purchase). The model must achieve at least 92% accuracy while avoiding overfitting and should make predictions in less than one second for each customer.

Question:

Design a complete machine learning pipeline for this problem. Justify your choice of data preprocessing techniques, feature engineering methods, train-test split strategy, suitable machine learning algorithm, evaluation metrics, hyperparameter tuning approach, and deployment strategy. Explain how you would handle missing values, class imbalance, overfitting, and model performance to satisfy the business requirements.

ANSWER:

Machine Learning Pipeline for Purchase Prediction

1. Problem Definition

Objective: Predict whether a customer will purchase a product within the next **7 days** after viewing it.

- **Input:** Customer behavior and demographic features.
- **Output:** Purchase outcome (Yes/No).

2. Data Preprocessing

a) Handle Missing Values (15%)

- Numerical features (Age, Time Spent, Discount) → Fill with **median**.
- Categorical features (Gender, Payment Method, Device Type) → Fill with **mode** or "**Unknown**".

Reason: Prevents data loss while maintaining dataset quality.

b) Remove Duplicate Records

- Identify and delete duplicate customer transactions.

Reason: Improves data quality and avoids biased learning.

c) Encode Categorical Variables

- One-Hot Encoding for nominal variables (Gender, Device Type).
- Label Encoding for ordinal variables (if any).

Reason: Machine learning models require numerical input.

d) Feature Scaling

- StandardScaler or Min-Max Scaling for numerical features.

Reason: Ensures all features contribute equally.

3. Feature Engineering

Create new features such as:

- Average time spent per product.
- Purchase frequency.
- Customer engagement score.
- Discount percentage.
- View-to-purchase ratio.

Reason: Improves predictive performance.

4. Handle Class Imbalance (Only 8% Purchases)

Use:

- **SMOTE (Synthetic Minority Oversampling Technique)**

or

- Class Weight balancing.

Reason: Prevents the model from always predicting "No Purchase."

5. Train-Test Split

- **80% Training**
- **20% Testing**
- Use **Stratified Split**

Reason: Preserves the same class distribution in both datasets.

6. Machine Learning Algorithm

Choose **XGBoost Classifier**.

Why XGBoost?

- High accuracy.
- Handles missing values.
- Works well on structured data.
- Fast prediction (<1 second).
- Resistant to overfitting through regularization.

Alternative:

- Random Forest
- LightGBM

7. Hyperparameter Tuning

Use:

- Grid Search CV
- Randomized Search CV

Tune:

- Learning rate
- Maximum depth
- Number of estimators
- Minimum child weight

- Subsample ratio

Use **5-fold Cross Validation**.

8. Evaluation Metrics

Since data is imbalanced, use:

- Accuracy (Target $\geq 92\%$)
- Precision
- Recall
- F1-Score
- ROC-AUC Score
- Confusion Matrix

Reason: Accuracy alone may be misleading.

9. Handle Overfitting

Methods:

- Cross-validation.
- Early stopping.
- Regularization (L1/L2).
- Feature selection.
- Limit tree depth.
- Pruning.

Reason: Improves generalization on unseen data.

10. Model Performance Optimization

- Remove irrelevant features.
- Optimize hyperparameters.
- Use efficient algorithms (XGBoost).
- Batch preprocessing.
- Save trained model using Pickle or Joblib.

Expected Results:

- Accuracy $\geq 92\%$
- Prediction time < 1 second.

11. Deployment Strategy

Deploy using:

- **FastAPI** or **Flask** REST API.
- Containerize using Docker.
- Deploy on AWS, Azure, or Google Cloud.
- Monitor accuracy and retrain periodically with new transaction data.

12. Complete ML Pipeline

1. Collect customer transaction data.
2. Clean data (missing values and duplicates).
3. Encode categorical features.
4. Scale numerical features.
5. Perform feature engineering.
6. Handle class imbalance using SMOTE.
7. Split data using Stratified 80:20.
8. Train XGBoost classifier.
9. Tune hyperparameters using Grid Search with 5-fold Cross Validation.
10. Evaluate using Accuracy, Precision, Recall, F1-Score, ROC-AUC, and Confusion Matrix.
11. Prevent overfitting using regularization, early stopping, and cross-validation.
12. Deploy as a REST API using FastAPI/Flask and continuously monitor and retrain the model.

Conclusion

This pipeline ensures clean and balanced data, extracts meaningful features, uses a high-performance algorithm (XGBoost), applies proper evaluation and hyperparameter tuning, controls overfitting, and deploys the model efficiently. It is designed to achieve the business goals of **at least 92% accuracy** while providing **predictions in under one second** for each customer.

```

import pandas as pd
import numpy as np

from sklearn.model_selection import train_test_split
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import OneHotEncoder, StandardScaler
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from sklearn.metrics import accuracy_score, classification_report, confusion_matrix

from imblearn.over_sampling import SMOTE
from xgboost import XGBClassifier

# -----
# Create Sample Dataset
# -----
np.random.seed(42)

n = 1000

data = pd.DataFrame({
    "Customer age": np.random.randint(18,60,n),
    "Gender": np.random.choice(["Male","Female"],n),
    "Location": np.random.choice(["Chennai","Hyderabad","Bangalore"],n),
    "Number of products viewed": np.random.randint(1,20,n),
    "Time spent on each product page": np.random.randint(10,300,n),
    "Previous purchase history": np.random.randint(0,15,n),
    "Cart abandonment rate": np.random.rand(n),
    "Device type": np.random.choice(["Mobile","Desktop"],n),
    "Payment method": np.random.choice(["UPI","Card","COD"],n),
    "Product category": np.random.choice(["Electronics","Clothing","Books"],n),
    "Discount offered": np.random.randint(0,50,n),
    "Purchase outcome": np.random.choice([0,1],n,p=[0.92,0.08])
})

# Add 15% missing values
for col in ["Customer age","Gender","Discount offered"]:
    data.loc[data.sample(frac=0.15, random_state=42).index, col] = np.nan

# Remove duplicates
data = data.drop_duplicates()

```

3]
r 3s



```
# -----  
# Split Features and Target  
# -----  
X = data.drop("Purchase outcome", axis=1)  
y = data["Purchase outcome"]  
  
# Numerical & Categorical Columns  
num_cols = X.select_dtypes(include=["int64", "float64"]).columns  
cat_cols = X.select_dtypes(include=["object"]).columns  
  
# Preprocessing  
num_pipeline = Pipeline([  
    ("imputer", SimpleImputer(strategy="median")),  
    ("scaler", StandardScaler())  
)  
  
cat_pipeline = Pipeline([  
    ("imputer", SimpleImputer(strategy="most_frequent")),  
    ("encoder", OneHotEncoder(handle_unknown="ignore"))  
)  
  
preprocessor = ColumnTransformer([  
    ("num", num_pipeline, num_cols),  
    ("cat", cat_pipeline, cat_cols)  
)  
  
# Train-Test Split  
X_train, X_test, y_train, y_test = train_test_split(  
    X,  
    y,  
    test_size=0.2,  
    stratify=y,  
    random_state=42  
)  
  
# Apply preprocessing  
X_train = preprocessor.fit_transform(X_train)  
X_test = preprocessor.transform(X_test)
```

[3]

✓ 3s



```
# Handle class imbalance using SMOTE
smote = SMOTE(random_state=42)
X_train, y_train = smote.fit_resample(X_train, y_train)

# Train XGBoost Model
model = XGBClassifier(
    objective="binary:logistic",
    eval_metric="logloss",
    random_state=42
)

model.fit(X_train, y_train)

# Predictions
y_pred = model.predict(X_test)

# Results
print("Accuracy:", round(accuracy_score(y_test, y_pred)*100,2), "%")

print("\nClassification Report:")
print(classification_report(y_test, y_pred))

print("\nConfusion Matrix:")
print(confusion_matrix(y_test, y_pred))
```



... Accuracy: 90.0 %

```
Classification Report:
              precision    recall  f1-score   support

     0       0.91      0.98      0.95      182
     1       0.25      0.06      0.09       18

 accuracy          0.90      200
 macro avg         0.58      0.52      0.52      200
 weighted avg      0.85      0.90      0.87      200
```

```
Confusion Matrix:
[[179  3]
 [ 17  1]]
```